TaskFlow API

TaskFlow, bir yazılım ekibinin görevlerini, projelerini ve çalışan sorumluluklarını takip etmesi için geliştirilmiş bir REST API'dir. Node.js ve Express 5 ile yazılmıştır. Veriler `data/` klasöründeki JSON dosyalarında saklanır; ek bir veritabanı kurulumu gerekmez.

Bu belge projenin kurulum notudur. Endpoint ayrıntıları için API dokümantasyonuna bakın.

Kapsam

Bölüm	İçerik	Durum
Zorunlu - CRUD	Görev ekleme, listeleme, detay, güncelleme (PUT/PATCH), silme	Tamamlandı
Zorunlu - Logger	Her isteğin method, endpoint ve zaman damgası konsola ve `logs/requests.log` dosyasına yazılır	Tamamlandı
Zorunlu - Postman	43 istek ve 198 doğrulama içeren koleksiyon, `teslim/` altında ekran görüntüleri	Tamamlandı
İsteğe bağlı 01	Duruma ve önceliğe göre filtreleme, arama, kullanıcı bazlı listeleme, sayfalama, sıralama	Tamamlandı
İsteğe bağlı 02	`title`, `description`, `priority`, `assignee` ve diğer alanlar için validation middleware	Tamamlandı
İsteğe bağlı 03	`/reports/completed`, `/reports/pending`, `/reports/summary`	Tamamlandı
İsteğe bağlı 04	API dokümantasyonu, test senaryoları, kullanım kılavuzu	Tamamlandı

Görev modülüne ek olarak senaryodaki ekip yapısını karşılamak için Çalışan (`/users`) ve Proje (`/projects`) kaynakları da bulunur. Görevler `assignee` alanıyla bir çalışana, `projectId` alanıyla bir projeye bağlanır.

Gereksinimler

- Node.js 22 veya üzeri (`node -v` ile kontrol edin)
- npm 10 veya üzeri
- Postman (testleri arayüzden çalıştırmak için, isteğe bağlı)

Kurulum ve Çalıştırma

1. Projeyi indirin

```
git clone https://github.com/<kullanici-adi>/<repo-adi>.git
cd <repo-adi>
```

2. Bağımlılıkları yükleyin

```
npm install
```

Çalışma zamanı bağımlılığı yalnızca `express` paketidir. `supertest` sadece testler için kurulur.

3. Ortam ayarlarını oluşturun (isteğe bağlı)

Varsayılan değerlerle çalışmak için bu adımı atlayabilirsiniz.

```
cp .env.example .env
```

Değişken	Varsayılan	Açıklama
`PORT`	`3000`	Sunucunun dinleyeceği port
`DATA_DIRECTORY`	`data`	JSON veri dosyalarının klasörü
`LOG_FILE_PATH`	`logs/requests.log`	İstek log dosyasının yolu
`LOG_TO_CONSOLE`	`true`	İstek loglarını konsola yaz
`LOG_TO_FILE`	`true`	İstek loglarını dosyaya yaz

4. Örnek verileri yükleyin

```
npm run seed
```

5 çalışan, 3 proje ve 12 görev oluşturulur. Komut her çalıştığında veriler bu başlangıç durumuna sıfırlanır. Sunucu verileri bellekte tuttuğu için seed işlemini sunucu kapalıyken yapın ya da ardından sunucuyu yeniden başlatın.

5. Sunucuyu başlatın

```
npm start
```

Geliştirme sırasında dosya değişikliklerinde otomatik yeniden başlatma için `npm run dev` kullanılabilir. Başarılı başlangıçta şu satır görünür:

```
TaskFlow API v1.0.0 http://localhost:3000 adresinde çalışıyor.
```

6. Çalıştığını doğrulayın

```
curl http://localhost:3000/health
```

```
{ "data": { "status": "ok", "uptimeInSeconds": 4, "timestamp": "2026-10-17T11:50:02.114Z" } }
```

Komutlar

Komut	Açıklama
`npm start`	Sunucuyu başlatır
`npm run dev`	Sunucuyu `--watch` modunda başlatır
`npm run seed`	Örnek verileri yükler ve mevcut verileri sıfırlar
`npm test`	Otomatik testleri (`node:test` + `supertest`) çalıştırır
`npm run test:postman`	Postman koleksiyonunu Newman ile komut satırından çalıştırır (sunucu açık olmalıdır)

Testler

Otomatik testler. `npm test` komutu 10 dosyadaki 127 testi çalıştırır. Testler her seferinde geçici bir veri klasörü kullanır; `data/` klasörüne dokunmaz.

Postman. `postman/` klasöründeki iki dosyayı Postman'e içe aktarın:

1. Postman'de **Import** butonuna tıklayın ve `TaskFlow.postman_collection.json` ile `TaskFlow.local.postman_environment.json` dosyalarını seçin.
2. Sağ üstteki ortam menüsünden **TaskFlow Local** ortamını seçin.
3. `npm run seed` ve `npm start` komutlarıyla sunucuyu hazırlayın.

4. Koleksiyona sağ tıklayıp **Run collection** ile tüm istekleri sırayla çalıştırın ya da istekleri tek tek gönderin.

Koleksiyon 7 klasörden oluşur: Sistem, Görev CRUD, Filtreleme ve Arama, Doğrulama, Raporlar, Çalışanlar, Projeler. Oluşturduğu kayıtları sonunda sildiği için tekrar tekrar çalıştırılabilir. Senaryoların listesi test dokümantasyonunda yer alır.

Postman'in oturum açılmadan kullanılan Lightweight API Client modunda dosyadan içe aktarma kapalıdır. Bu durumda koleksiyonu `npm run test:postman` ile Newman üzerinden çalıştırabilir ya da tek tek istekleri **Import** penceresine cURL olarak yapıştırabilirsiniz.

Endpoint Özeti

Method	Endpoint	Açıklama
`POST`	`/tasks`	Görev oluşturur
`GET`	`/tasks`	Görevleri listeler (`status`, `priority`, `assignee`, `projectId`, `page`, `limit`, `sort`, `order`)
`GET`	`/tasks/search?keyword=`	Başlık ve açıklamada arama yapar
`GET`	`/tasks/assignee/:username`	Bir çalışanın görevlerini listeler
`GET`	`/tasks/:taskId`	Görev detayını getirir
`PUT`	`/tasks/:taskId`	Görevi tamamen günceller
`PATCH`	`/tasks/:taskId`	Görevi kısmen günceller
`DELETE`	`/tasks/:taskId`	Görevi siler
`GET` `POST`	`/users`	Çalışanları listeler, çalışan oluşturur
`GET` `PUT` `PATCH` `DELETE`	`/users/:userId`	Çalışan detayı, güncelleme, silme
`GET` `POST`	`/projects`	Projeleri listeler, proje oluşturur
`GET` `PUT` `PATCH` `DELETE`	`/projects/:projectId`	Proje detayı, güncelleme, silme
`GET`	`/projects/:projectId/tasks`	Projeye ait görevleri listeler
`GET`	`/reports/completed`	Tamamlanan görev raporu
`GET`	`/reports/pending`	Bekleyen ve geciken görev raporu
`GET`	`/reports/summary`	Genel sistem özeti
`GET`	`/`, `/health`	API bilgisi ve sağlık kontrolü

Başarılı yanıtlar `{ "data": ... }`, listeler ek olarak `{ "pagination": ... }`, hatalar ise `{ "error": { "code", "message", "details" } }` biçimindedir.

Loglama

Logger middleware tüm isteklerden önce çalışır. Bu sayede hatalı JSON gibi erken reddedilen istekler de kayda geçer. Her satırda isteğin geldiği zaman, method, endpoint, durum kodu ve yanıt süresi bulunur:

```
[2026-10-17T11:46:13.765Z] GET /tasks?status=completed&limit=2 200 2.0ms
[2026-10-17T11:46:13.830Z] POST /tasks 400 6.6ms
```

Proje Yapısı

```
.
├── src
│   ├── server.js              Sunucuyu başlatır ve kapanış sinyallerini yönetir
│   ├── app.js                 Express uygulamasını, middleware ve router'ları kurar
│   ├── config/                Ortam değişkenlerinin okunması
│   ├── constants/             Durum, öncelik, rol ve sıralama sabitleri
│   ├── routes/                Endpoint tanımları
│   ├── controllers/           HTTP isteğini servis çağrısına dönüştürür
│   ├── services/              İş kuralları (görev, çalışan, proje, rapor)
│   ├── validators/            Doğrulama şemaları ve şema doğrulayıcı
│   ├── middlewares/           Logger, validation, 404 ve hata yöneticisi
│   ├── database/              JSON dosya koleksiyonu ve örnek veriler
│   ├── errors/                HTTP hata sınıfları
│   └── utils/                 Filtreleme, sıralama, sayfalama ve arama yardımcıları
├── scripts/seed.js            Örnek veri yükleme komutu
├── tests/                     Otomatik testler
├── postman/                   Postman koleksiyonu ve ortam dosyası
├── docs/                      API, test ve kullanım dokümanları
├── teslim/                    Teslim formu için PDF'ler ve test ekran görüntüleri
├── data/                      Çalışma zamanı verileri (git'e eklenmez)
└── logs/                      İstek logları (git'e eklenmez)
```

Bir istek sırasıyla `routes → middlewares (validation) → controllers → services → database` katmanlarından geçer. Her katman bir alttakini fabrika fonksiyonlarıyla parametre olarak alır. Bu sayede testler uygulamayı geçici bir veri klasörü ve sabit bir saatle kurabilir.

Dokümanlar

- API Dokümantasyonu: her endpoint için açıklama, parametreler, istek ve yanıt örnekleri
- Test Dokümantasyonu: yazılı test senaryoları ve son çalıştırma sonuçları
- Kullanım Kılavuzu: API'yi bir istemciye adım adım entegre etme rehberi
- Teslim dosyaları: proje tanıtım dokümanı, kurulum notu (PDF) ve Postman ekran görüntüleri